МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КУБГУ»)

Кафедра вычислительных технологий

Отчёт

по лабораторной работе №2

по дисциплине «Методы разработки трансляторов»

на тему: «Перевод исходной программы в

обратную польскую запись»

Работу выполнил

студент группы

ФИ 31/2

Костров А.А.

_____

(подпись)

Преподаватель:

Усов П.Е.

_____

(подпись)

Краснодар
2026

Цель работы

Изучить алгоритм Дейкстры и реализовать программу перевода выражений языка R в обратную польскую запись.

Теоретическая часть

Обратная польская запись (ОПЗ) — форма записи выражений, в которой операнды располагаются перед оператором. Преимущество ОПЗ заключается в возможности вычисления выражения за один проход.

Алгоритм Дейкстры использует стек для обработки операторов с учётом их приоритетов.

Описание программы

Программа состоит из двух основных модулей:
1. Лексический анализатор (lexer.py), который разбивает исходный код на лексемы.
2. Модуль перевода в ОПЗ (polska.py), реализующий алгоритм Дейкстры.

Лексический анализатор формирует таблицы идентификаторов, чисел, строк и операций.
Конвертер ОПЗ обрабатывает полученные токены, используя стек и таблицу приоритетов.

Основные возможности программы

- Поддержка арифметических и логических операций
- Обработка функций и массивов
- Поддержка условных операторов (if, else)
- Поддержка циклов (while)
- Формирование ОПЗ строки

Пример

Исходное выражение:
(a + b) * c

ОПЗ:
a b + c *

Алгоритм перевода в ОПЗ (подробно)

Алгоритм перевода выражений в обратную польскую запись основан на алгоритме Дейкстры (shunting-yard).

Основные шаги алгоритма:

1. Инициализируется пустой стек операторов и выходная строка.
2. Входное выражение обрабатывается слева направо по одному токену.
3. Если токен является операндом (число, идентификатор), он сразу добавляется в выход.
4. Если токен является оператором:
   - пока в стеке есть оператор с большим или равным приоритетом, он извлекается в выход;
   - текущий оператор помещается в стек.
5. Если токен — открывающая скобка, он помещается в стек.
6. Если токен — закрывающая скобка:
   - из стека извлекаются операторы до ближайшей открывающей скобки.
7. После обработки всех токенов оставшиеся операторы извлекаются из стека в выход.

Дополнительно:

- Для функций ведётся счётчик аргументов.
- Для массивов используется операция АЭМ.
- Для условных операторов формируются метки переходов.

Разбор реализации в файле polska.py

Класс OPZConverter реализует расширенный алгоритм Дейкстры с поддержкой конструкций языка R.

Основные структуры:

- stack — стек операторов
- output — выходная строка (ОПЗ)
- tokens — входные токены
- history — история шагов алгоритма

1. Операнды:
   - Если токен является операндом (число, строка, идентификатор), он добавляется в output.

2. Операторы:
   - Сравнивается приоритет с вершиной стека
   - При необходимости операторы извлекаются в output
   - Текущий оператор помещается в стек

3. Скобки:
   - '(' помещается в стек
   - ')' извлекает операторы до '('

4. Функции:
   - При встрече идентификатора + '(' добавляется оператор 'Ф'
   - Ведётся подсчёт аргументов
   - При закрытии формируется запись вида: аргументы + k Ф

5. Массивы:
   - '[' инициирует операцию АЭМ
   - ',' увеличивает счётчик аргументов
   - ']' завершает АЭМ

6. Условные операторы:
   - if добавляет маркер IF в стек
   - then формирует условный переход (УПЛ)
   - else формирует безусловный переход (БП) и метки

7. Циклы while:
   - добавляются метки начала и конца
   - используется оператор УПЛ

Приоритеты операций

Приоритеты задаются в словаре PRIORITIES.

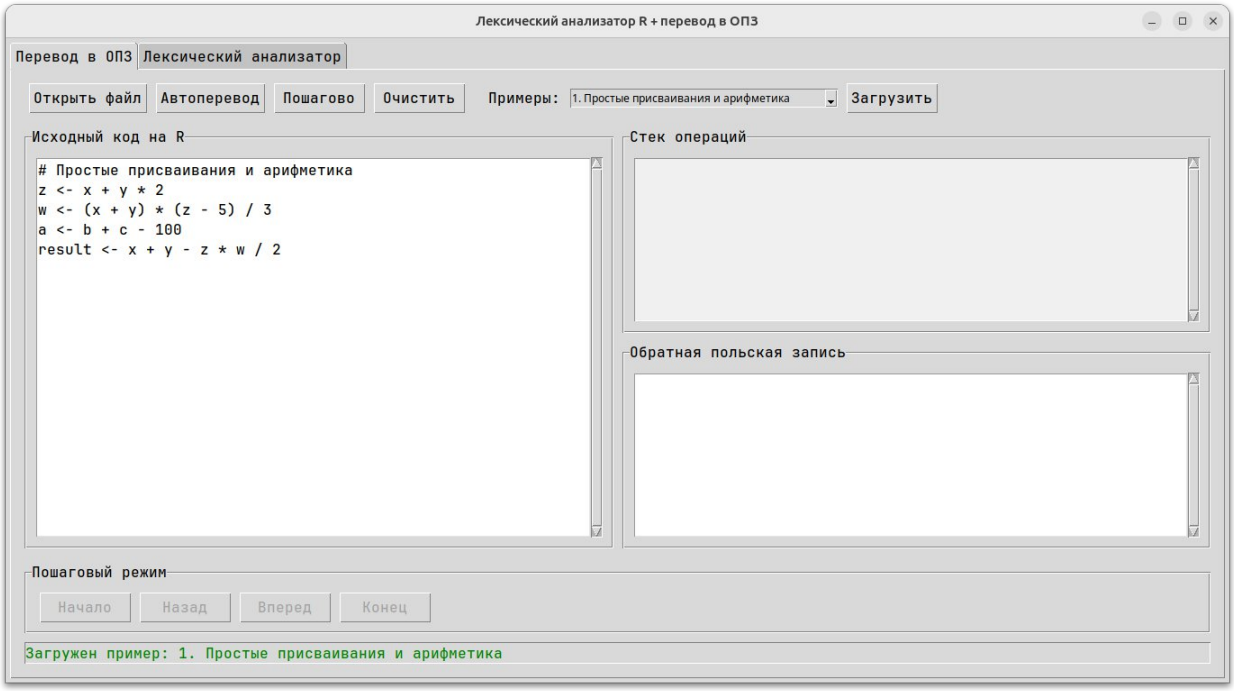
Чем больше значение — тем выше приоритет.

Например:

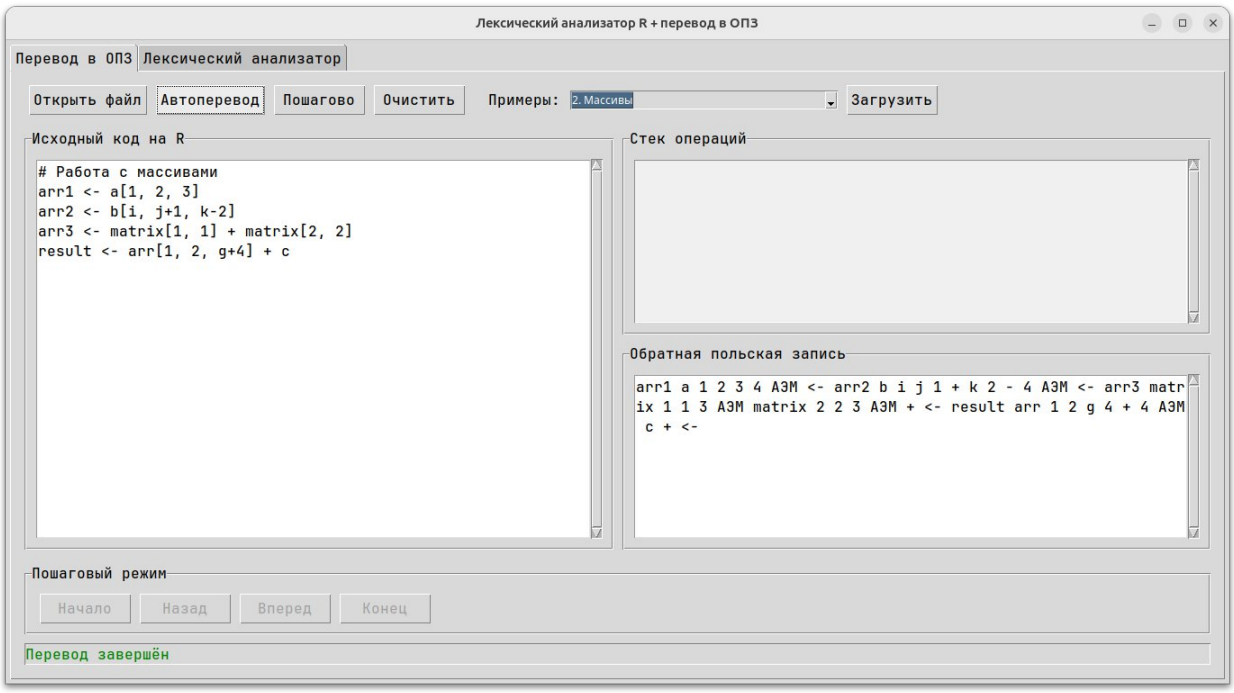
- +, - имеют приоритет 7
- *, / — 8
- ^ — 9
- логические операции имеют более низкий приоритет

Программа

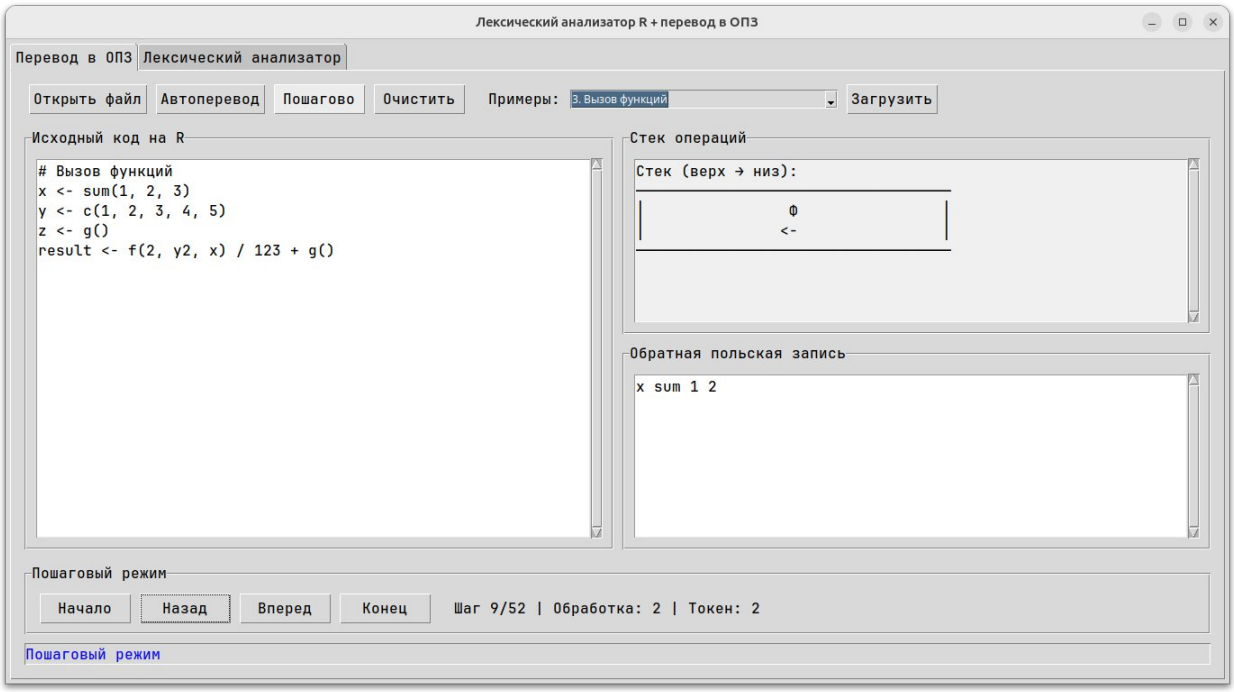
Программа представляет собой приложение с графическим интерфейсом. В левой части окна находится панель ввода исходного кода на языке R, а справа находятся результаты перевода в обратную польскую запись и стек операций.



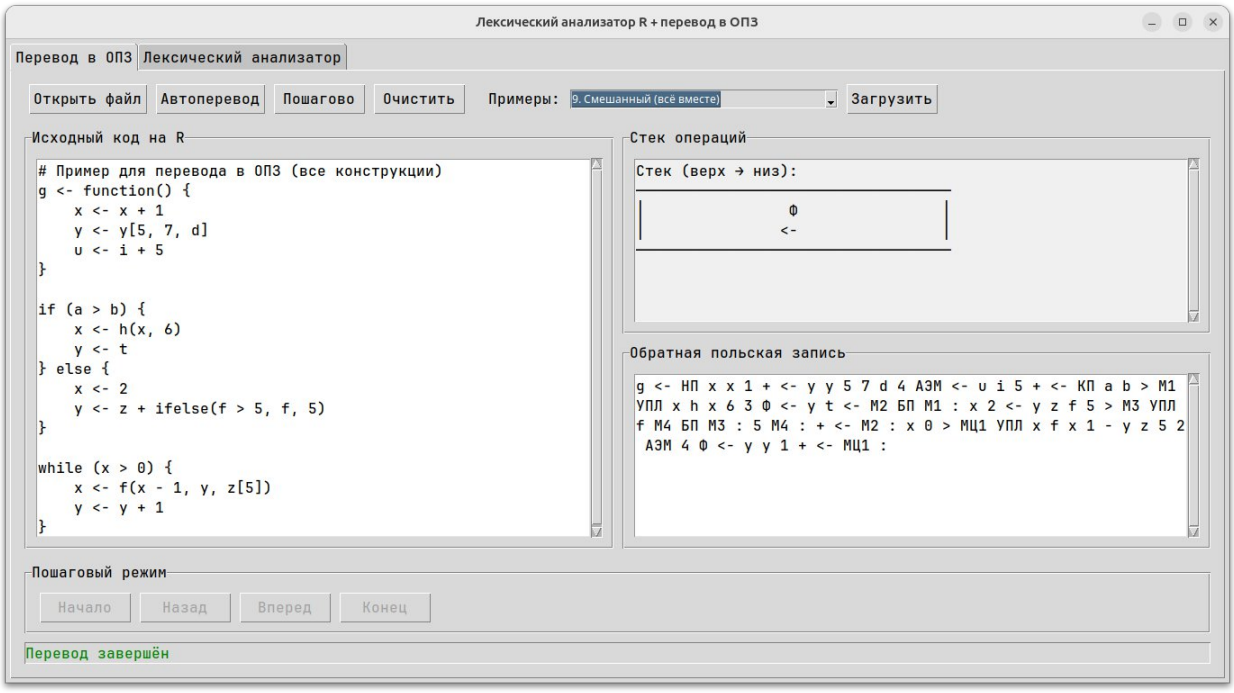
Программа поддерживает режим автоматического перевода в ОПЗ.



Помимо автоперевода программа предоставляет пользователю возможность запустить алгоритм перевода в пошаговом режиме.



Пользователь может выбрать какой-либо из примеров для демонстрации работы программы.



Структура компонентов программы

Класс OPZConverter

Основной класс, реализующий алгоритм Дейкстры с расширениями.

Таблица приоритетов

```
PRIORITIES = {
    # Открывающие конструкции (приоритет 0)
    '(': 0, '[': 0, 'АЭМ': 0, 'Ф': 0, 'IF': 0, 'WHILE': 0, 'FUNCTION': 0,

    # Закрывающие конструкции (приоритет 1)
    ')': 1, ']': 1, 'THEN': 1, 'ELSE': 1, ';': 1, '\n': 1,

    # Присваивание (приоритет 2)
    '=': 2, '<-': 2, '<<-': 2, '->': 2, '->>': 2, ':=': 2,

    # Логические операции (приоритеты 3-5)
    '||': 3, '|': 3, '&&': 4, '&': 4, '!': 5,

    # Отношения (приоритет 6)
    '<': 6, '>': 6, '<=': 6, '>=': 6, '==': 6, '!=': 6,

    # Арифметика (приоритеты 7-9)
    '+': 7, '-': 7, '*': 8, '/': 8, '^': 9, '**': 9,
}
```

Основные атрибуты состояния

Атрибут	*Назначение*
stack	*Стек операций (аналог стека алгоритма Дейкстры)*
output	*Выходная строка ОПЗ*
label_counter	*Счётчик для генерации уникальных меток*
array_depth / array_operand_count	*Обработка операции АЭМ*
func_depth / func_arg_count	*Обработка операции Ф*
if_depth / if_labels	*Обработка условных конструкций*
while_depth / while_labels	*Обработка циклов*

Реализация ключевых алгоритмов

Обработка массивов (операция АЭМ)

```
if value == '[':
    self.stack.append('АЭМ')
    self.array_operand_count.append(2)  # имя_массива + первый_индекс
    self.array_depth += 1

if value == ']':
    # Выталкиваем всё до АЭМ
    while self.stack and self.stack[-1] != 'АЭМ':
        self.output.append(self.stack.pop())
    # Добавляем операцию АЭМ со счётчиком
    count = self.array_operand_count.pop()
    self.output.append(str(count))
    self.output.append('АЭМ')

if value == ',' and self.array_depth > 0:
    # Запятая наращивает счётчик операндов
    self.array_operand_count[-1] += 1
```

Пример перевода: a[i+20, j] → a i 20 + j 3АЭМ

Обработка функций (операция Ф)

```
if value == '(' and self.last_was_identifier:
    self.stack.append('Ф')
    self.func_arg_count.append(1)  # имя_функции считается операндом

if value == ')':
    if self.func_depth > 0:
        while self.stack and self.stack[-1] != 'Ф':
            self.output.append(self.stack.pop())
        count = self.func_arg_count.pop() + 1
        self.output.append(str(count))
        self.output.append('Ф')
```

Пример: f(x, z, y+2) → f x z y 2 + 4Ф

Обработка условного оператора if-else

```
# IF заносится в стек с приоритетом 0
if value_lower == 'if':
    self.stack.append('IF')
    self.if_depth += 1

# THEN генерирует условный переход
def handle_then(self):
    while self.stack and self.stack[-1] != 'IF':
        self.output.append(self.stack.pop())
    mi = f"M{self.label_counter}"
    self.label_counter += 1
    self.output.append(mi)
    self.output.append('УПЛ')

# ELSE генерирует безусловный переход и метку
if value_lower == 'else':
    m_end = f"M{self.label_counter}"
    self.output.append(m_end)
    self.output.append('БП')
    self.output.append(m_else)
    self.output.append(':')
```

Формат ОПЗ для условного оператора:

условие M1 УПЛ оператор1 M2 БП M1: оператор2 M2:

Обработка цикла while

Расширение алгоритма для поддержки циклов (аналогично условному оператору):

```
if value_lower == 'while':
    self.stack.append('WHILE')
    self.while_depth += 1

# При обработке '(' после while генерируется метка цикла
if value == ')' and self.while_depth > 0:
    m_while = f"МЦ{self.while_counter}"
    self.output.append(m_while)
    self.output.append('УПЛ')
```

Пошаговая визуализация

Программа сохраняет историю состояний для пошагового просмотра:

```
def _save_state(self, action, token_value):
    state = {
        'step': len(self.history),
        'action': action,
        'stack': self.stack.copy(),
        'output': self.output.copy(),
        'current_token': token_value,
    }
    self.history.append(state)
```

Это позволяет:

Просматривать процесс перевода по шагам
Анализировать состояние стека на каждом этапе
Отлаживать алгоритм

Полный текст разработанной программы:

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox, scrolledtext
from pathlib import Path

from lexer import RLexer, LexemType


class OPZConverter:
    """Класс для перевода в ОПЗ с поддержкой while и function"""

    PRIORITIES = {
        '(': 0, '[': 0, 'АЭМ': 0, 'Ф': 0, 'IF': 0, 'WHILE': 0, 'FUNCTION': 0,
        ')': 1, ']': 1, 'THEN': 1, 'ELSE': 1, ';': 1, '\n': 1,
        '=': 2, '←': 2, '↞': 2, '->': 2, '->>': 2, ':=': 2,
        '||': 3, '|': 3, '&&': 4, '&': 4, '!': 5,
        '<': 6, '>': 6, '⩽': 6, '⩾': 6, '⩵': 6, '≠': 6,
        '+': 7, '-': 7, '*': 8, '/': 8, '%%': 8, '%/%': 8,
        '^': 9, '**': 9, ':': 10, '?': 10, '~': 10, '$': 11, '@': 11,
    }

    CONTROL_KEYWORDS = {'if', 'else', 'while', 'for', 'function', 'return', 'next', 'break'}

    def __init__(self):
        self.reset_all()

    def reset_all(self):
        self.stack = []
        self.output = []
        self.history = []
        self.current_step = -1
        self.tokens = []
        self.current_pos = 0
        self.array_depth = 0
        self.array_operand_count = []
        self.func_depth = 0
        self.func_arg_count = []
```

```
        self.last_was_identifier = False
        self.if_depth = 0
        self.else_encountered = False
        self.label_counter = 1
        self.if_labels = []
        self.block_depth = 0
        self.tokens_ahead = []
        self.in_ifelse = False
        self.ifelse_commas = 0
        self.in_while = False
        self.while_labels = []
        self.while_depth = 0
        self.in_function = False
        self.function_depth = 0
        self.func_name = None
        self.waiting_for_func_name = False
        self.while_counter = 1

    def get_priority(self, op):
        return self.PRIORITIES.get(op, 0)

    def is_operator(self, token):
        return token.lex_type == LexemType.OPERATION

    def is_operand(self, token):
        if token.lex_type in (LexemType.IDENTIFIER, LexemType.NUMBER, LexemType.STRING):
            return True
        if token.lex_type == LexemType.KEYWORD:
            if token.value not in self.CONTROL_KEYWORDS and token.value != 'ifelse':
                return True
        return False

    def _save_state(self, action, token_value):
        display_token = token_value.replace('\n', '\\n').replace('\r', '\\r') if token_value else
""
        state = {
            'step': len(self.history),
            'action': action,
            'stack': self.stack.copy(),
            'output': self.output.copy(),
            'current_token': display_token,
        }
        self.history.append(state)

    def load_history(self, history):
        self.history = history
        self.current_step = 0

    def convert_expression(self, tokens):
        self.reset_all()
        self.tokens = tokens
        self.tokens_ahead = tokens

        self._save_state("Начало", "")

        i = 0
        while i < len(tokens):
            token = tokens[i]
            self.current_pos = i

            next_tokens = tokens[i+1:] if i+1 < len(tokens) else []

            self._process_token(token, next_tokens)
            if token.value == '\n':
                self._save_state(f"Обработка: \\n", token.value)
            else:
                self._save_state(f"Обработка: {token.value}", token.value)
            i += 1
```

```
        self._close_pending_ifs()
        while self.stack:
            op = self.stack.pop()
            if op not in ('(', ')'):
                self.output.append(op)

        self._save_state("Завершение", "")
        return ' '.join(self.output), self.history

    def _has_else_ahead(self, next_tokens):
        for t in next_tokens:
            if t.value.lower() == 'else':
                return True
            if t.value == '{':
                continue
            if t.value not in ('\n', '\r', ' ', ';'):
                break
        return False

    def _handle_if_close(self):
        if self.if_depth > 0 and any(x == 'IF' for x in self.stack):
            while self.stack and self.stack[-1] != 'IF':
                op = self.stack.pop()
                if op not in ('(', 'АЭМ', 'Ф', '['):
                    self.output.append(op)
            if self.stack and self.stack[-1] == 'IF':
                self.stack.pop()
                if self.if_labels:
                    m = self.if_labels.pop()
                    self.output.append(m)
                    self.output.append(':')
                self.if_depth -= 1
                self.else_encountered = False

    def _close_pending_ifs(self):
        while self.if_depth > 0:
            self._handle_if_close()

    def handle_then(self):
        while self.stack and self.stack[-1] != 'IF':
            op = self.stack.pop()
            if op not in ('(', 'АЭМ', 'Ф', '['):
                self.output.append(op)
        if self.stack and self.stack[-1] == 'IF':
            mi = f"M{self.label_counter}"
            self.label_counter += 1
            self.if_labels.append(mi)
            self.output.append(mi)
            self.output.append('УПЛ')

    def flush_stack_to_output(self, until=None):
        while self.stack:
            top = self.stack[-1]
            if until and top == until:
                break
            if top not in ('(', 'АЭМ', 'Ф', 'IF', 'WHILE', 'FUNCTION', '['):
                op = self.stack.pop()
                self.output.append(op)
            else:
                break

    def _process_token(self, token, next_tokens=None):
        value = token.value
        lex_type = token.lex_type
        value_lower = value.lower() if isinstance(value, str) else value

        if value == ';':
```

```
            if self.block_depth > 0:
                while self.stack and self.stack[-1] not in ('(', '[', '{', 'IF', 'WHILE',
'FUNCTION'):
                    op = self.stack.pop()
                    if op not in ('(', '[', '{', 'IF', 'WHILE', 'FUNCTION'):
                        self.output.append(op)
            else:
                self._handle_if_close()
                self.flush_stack_to_output()
            self.array_depth = 0
            self.array_operand_count = []
            self.func_depth = 0
            self.func_arg_count = []
            self.last_was_identifier = False
            return

        if value == '\n' or value == '\r':
            if self.block_depth > 0:
                while self.stack and self.stack[-1] not in ('(', '[', '{', 'IF', 'WHILE',
'FUNCTION'):
                    op = self.stack.pop()
                    if op not in ('(', '[', '{', 'IF', 'WHILE', 'FUNCTION'):
                        self.output.append(op)
            else:
                if not self.in_ifelse:
                    self._handle_if_close()
                    self.flush_stack_to_output()
                    self.array_depth = 0
                    self.array_operand_count = []
                    self.func_depth = 0
                    self.func_arg_count = []
                    self.last_was_identifier = False
            return

        if value_lower == 'function':
            while self.stack and self.stack[-1] in ('←', '«←', '->', '->>', ':=', '='):
                op = self.stack.pop()
                self.output.append(op)
            self.function_depth += 1
            self.last_was_identifier = False
            return

        if value_lower == 'while':
            self.stack.append('WHILE')
            self.while_depth += 1
            self.in_while = True
            self.last_was_identifier = False
            return

        if value_lower == 'ifelse':
            self.stack.append('IF')
            self.if_depth += 1
            self.else_encountered = False
            self.in_ifelse = True
            self.ifelse_commas = 0
            self.last_was_identifier = False
            return

        if value_lower == 'if':
            self.stack.append('IF')
            self.if_depth += 1
            self.else_encountered = False
            self.last_was_identifier = False
            return

        if value_lower == 'then':
            self.handle_then()
            self.last_was_identifier = False
```

```
        return

    if value_lower == 'else':
        if self.if_depth > 0 and not self.else_encountered:
            while self.stack and self.stack[-1] != 'IF':
                op = self.stack.pop()
                if op not in ('(', '[', '{'):
                    self.output.append(op)

            m_end = f"M{self.label_counter}"
            self.label_counter += 1
            self.output.append(m_end)
            self.output.append('БП')

            if self.if_labels:
                m_else = self.if_labels.pop()
            else:
                m_else = f"M{self.label_counter}"
                self.label_counter += 1

            self.output.append(m_else)
            self.output.append(':')
            self.if_labels.append(m_end)
            self.else_encountered = True
        else:
            while self.stack and self.stack[-1] != 'IF':
                op = self.stack.pop()
                if op not in ('(', '[', '{'):
                    self.output.append(op)

        self.last_was_identifier = False
        return

    if self.is_operand(token):
        if self.waiting_for_func_name:
            self.output.append(value)
            self.output.append('НП')
            self.waiting_for_func_name = False
        else:
            self.output.append(value)
        self.last_was_identifier = True
        return

    if lex_type == LexemType.COMMENT:
        return

    if value == '{':
        if self.function_depth > 0 and self.block_depth == 0:
            self.output.append('НП')
        self.stack.append('{')
        self.block_depth += 1
        self.last_was_identifier = False
        return

    if value == '}':
        while self.stack and self.stack[-1] != '{':
            op = self.stack.pop()
            if op not in ('(', '['):
                self.output.append(op)
        if self.stack and self.stack[-1] == '{':
            self.stack.pop()
        self.block_depth -= 1

        if self.block_depth == 0 and self.while_depth > 0:
            while self.stack and self.stack[-1] != 'WHILE':
                op = self.stack.pop()
                if op not in ('(', '['):
                    self.output.append(op)
```

```
                if self.stack and self.stack[-1] == 'WHILE':
                    self.stack.pop()

                if self.while_labels:
                    m_end = self.while_labels.pop()
                    self.output.append(m_end)
                    self.output.append(':')
                self.while_depth -= 1
                self.in_while = False

            if self.block_depth == 0 and self.function_depth > 0:
                self.output.append('КП')
                self.function_depth -= 1

            if self.block_depth == 0 and self.if_depth > 0 and not self.else_encountered:
                if next_tokens and self._has_else_ahead(next_tokens):
                    pass
                else:
                    self._handle_if_close()
            elif self.block_depth == 0 and self.if_depth > 0 and self.else_encountered:
                self._handle_if_close()

            self.last_was_identifier = False
            return

        if value == '(':
            if self.last_was_identifier:
                self.stack.append('Ф')
                self.func_arg_count.append(1)
                self.func_depth += 1
            else:
                self.stack.append('(')
            self.last_was_identifier = False
            return

        if value == ')':
            if self.in_ifelse:
                while self.stack and self.stack[-1] != 'IF':
                    op = self.stack.pop()
                    if op not in ('(', '[', '{'):
                        self.output.append(op)
                if self.stack and self.stack[-1] == 'IF':
                    self.stack.pop()
                    if self.if_labels:
                        m_end = self.if_labels.pop()
                        self.output.append(m_end)
                        self.output.append(':')
                    self.if_depth -= 1
                self.in_ifelse = False
                self.ifelse_commas = 0
                self.last_was_identifier = False
                return

            if self.func_depth > 0:
                while self.stack and self.stack[-1] != 'Ф':
                    op = self.stack.pop()
                    if op not in ('(', 'АЭМ', '['):
                        self.output.append(op)
                if self.stack and self.stack[-1] == 'Ф':
                    self.stack.pop()
                    count = self.func_arg_count.pop() + 1 if self.func_arg_count and
self.func_arg_count[-1] > 1 else 1
                    self.output.append(str(count))
                    self.output.append('Ф')
                self.func_depth -= 1
                self.last_was_identifier = False
                return
```

```
            if self.while_depth > 0 and any(x == 'WHILE' for x in self.stack):
                while self.stack and self.stack[-1] != 'WHILE':
                    op = self.stack.pop()
                    if op not in ('(', '['):
                        self.output.append(op)
                m_while = f"МЦ{self.while_counter}"
                self.while_counter += 1
                self.while_labels.append(m_while)
                self.output.append(m_while)
                self.output.append('УПЛ')
                self.last_was_identifier = False
                return

            while self.stack and self.stack[-1] not in ('(', 'IF'):
                op = self.stack.pop()
                if op not in ('АЭМ', 'Ф', '['):
                    self.output.append(op)
            if self.stack and self.stack[-1] == '(':
                self.stack.pop()

            if self.if_depth > 0 and any(x == 'IF' for x in self.stack) and not
self.else_encountered and not self.in_ifelse:
                self.handle_then()

            self.last_was_identifier = False
            return

        if value == '[':
            self.stack.append('АЭМ')
            self.array_operand_count.append(2)
            self.array_depth += 1
            self.last_was_identifier = False
            return

        if value == ']':
            while self.stack and self.stack[-1] != 'АЭМ':
                op = self.stack.pop()
                if op != '[':
                    self.output.append(op)
            if self.stack and self.stack[-1] == 'АЭМ':
                self.stack.pop()
                count = self.array_operand_count.pop() if self.array_operand_count else 2
                self.output.append(str(count))
                self.output.append('АЭМ')
            self.array_depth -= 1
            self.last_was_identifier = False
            return

        if value == ',':
            if self.in_ifelse:
                self.ifelse_commas += 1
                if self.ifelse_commas == 1:
                    while self.stack and self.stack[-1] != 'IF':
                        op = self.stack.pop()
                        if op not in ('(', '[', '{'):
                            self.output.append(op)
                    if self.stack and self.stack[-1] == 'IF':
                        mi = f"M{self.label_counter}"
                        self.label_counter += 1
                        self.if_labels.append(mi)
                        self.output.append(mi)
                        self.output.append('УПЛ')
                elif self.ifelse_commas == 2:
                    while self.stack and self.stack[-1] != 'IF':
                        op = self.stack.pop()
                        if op not in ('(', '[', '{'):
                            self.output.append(op)
```

```
                    m_end = f"M{self.label_counter}"
                    self.label_counter += 1
                    self.output.append(m_end)
                    self.output.append('БП')

                    if self.if_labels:
                        m_else = self.if_labels.pop()
                    else:
                        m_else = f"M{self.label_counter}"
                        self.label_counter += 1

                    self.output.append(m_else)
                    self.output.append(':')
                    self.if_labels.append(m_end)
                self.last_was_identifier = False
                return

            if self.array_depth > 0:
                while self.stack and self.stack[-1] != 'АЭМ':
                    self.output.append(self.stack.pop())
                if self.array_operand_count:
                    self.array_operand_count[-1] += 1
            elif self.func_depth > 0:
                while self.stack and self.stack[-1] != 'Ф':
                    self.output.append(self.stack.pop())
                if self.func_arg_count:
                    self.func_arg_count[-1] += 1
            elif self.if_depth > 0:
                if not self.else_encountered:
                    self.handle_then()
                    self.else_encountered = True
            else:
                while self.stack and self.stack[-1] != '(':
                    self.output.append(self.stack.pop())
            self.last_was_identifier = False
            return

        if self.is_operator(token):
            priority = self.get_priority(value)
            while (self.stack and self.stack[-1] not in ('(', 'АЭМ', 'Ф', 'IF', 'WHILE',
'FUNCTION', '[') and
                   self.get_priority(self.stack[-1]) >= priority):
                self.output.append(self.stack.pop())
            self.stack.append(value)
            self.last_was_identifier = False
            return

        if value in ('←', '⇐', '->', '->>', ':=', '='):
            priority = self.get_priority(value)
            while (self.stack and self.stack[-1] not in ('(', 'АЭМ', 'Ф', 'IF', 'WHILE',
'FUNCTION', '[') and
                   self.get_priority(self.stack[-1]) >= priority):
                self.output.append(self.stack.pop())
            self.stack.append(value)
            self.last_was_identifier = False
            return

        if lex_type == LexemType.DELIMITER:
            self.output.append(value)
            self.last_was_identifier = False
            return

    def step_forward(self):
        if self.current_step < len(self.history) - 1:
            self.current_step += 1
            return self.history[self.current_step]
        return None
```

```
    def step_backward(self):
        if self.current_step > 0:
            self.current_step -= 1
            return self.history[self.current_step]
        return None

    def reset_to_start(self):
        self.current_step = 0
        return self.history[0] if self.history else None

    def go_to_end(self):
        self.current_step = len(self.history) - 1
        return self.history[-1] if self.history else None

    def get_current_state(self):
        if 0 ≤ self.current_step < len(self.history):
            return self.history[self.current_step]
        return None


class SharedData:
    def __init__(self):
        self.tokens = []
        self.errors = []
        self.identifiers = {}
        self.numbers = {}
        self.strings = {}
        self.comments = {}
        self.keywords = {}
        self.delimiters = {}
        self.operations = {}


class LexicalAnalyzerGUI:
    def __init__(self, parent, shared, default_font):
        self.parent = parent
        self.shared = shared
        self.default_font = default_font
        self.setup_ui()

    def setup_ui(self):
        main = ttk.Frame(self.parent, padding="15")
        main.pack(fill=tk.BOTH, expand=True)
        main.columnconfigure(0, weight=1)
        main.rowconfigure(0, weight=1)

        notebook = ttk.Notebook(main)
        notebook.grid(row=0, column=0, sticky="nsew")

        tf = ttk.Frame(notebook)
        notebook.add(tf, text="Лексемы")
        tf.columnconfigure(0, weight=1)
        tf.rowconfigure(0, weight=1)
        self.tokens_text = scrolledtext.ScrolledText(tf, font=self.default_font)
        self.tokens_text.grid(row=0, column=0, sticky="nsew")

        tf2 = ttk.Frame(notebook)
        notebook.add(tf2, text="Таблицы")
        tf2.columnconfigure(0, weight=1)
        tf2.rowconfigure(0, weight=1)
        self.tables_text = scrolledtext.ScrolledText(tf2, font=self.default_font)
        self.tables_text.grid(row=0, column=0, sticky="nsew")

        ef = ttk.Frame(notebook)
        notebook.add(ef, text="Ошибки")
        ef.columnconfigure(0, weight=1)
        ef.rowconfigure(0, weight=1)
```

```
        self.errors_text = scrolledtext.ScrolledText(ef, font=self.default_font, bg='#fff0f0',
fg='red')
        self.errors_text.grid(row=0, column=0, sticky="nsew")

        sf = ttk.LabelFrame(main, text="Статистика", padding="10")
        sf.grid(row=1, column=0, sticky="ew", pady=(15, 0))

        self.stats_labels = {}
        items = [("Всего лексем:", 0), ("Идентификаторов:", 2), ("Чисел:", 4),
                 ("Строк:", 6), ("Ключевых слов:", 8), ("Операций:", 10), ("Ошибок:", 12)]

        bold_font = (self.default_font[0], self.default_font[1], 'bold')
        for i, (label, col) in enumerate(items):
            ttk.Label(sf, text=label, font=bold_font).grid(row=0, column=col, sticky='w',
padx=(20 if i > 0 else 0, 5))
            self.stats_labels[label] = ttk.Label(sf, text="0", foreground="blue", font=bold_font)
            self.stats_labels[label].grid(row=0, column=col + 1, sticky='w', padx=5)

    def update(self):
        self._update_tokens()
        self._update_tables()
        self._update_errors()
        self._update_stats()

    def _update_tokens(self):
        self.tokens_text.delete(1.0, tk.END)
        if not self.shared.tokens:
            self.tokens_text.insert(1.0, "Нет данных. Выполните перевод.")
            return
        self.tokens_text.insert(1.0, "ПОСЛЕДОВАТЕЛЬНОСТЬ ЛЕКСЕМ\n" + "=" * 90 + "\n")
        self.tokens_text.insert(tk.END, f"{'№':4s} {'Код':8s} {'Значение':30s} {'Стр':4s}
{'Статус':10s}\n")
        self.tokens_text.insert(tk.END, "-" * 90 + "\n")
        for i, t in enumerate(self.shared.tokens[:200], 1):
            status = "ОШИБКА" if t.lex_type == LexemType.ERROR else "ОК"
            val = t.value.replace('\n', '\\n').replace('\r', '\\r')
            self.tokens_text.insert(tk.END, f"{i:4d} {t.code:8s} {val[:30]:30s} {t.line:4d}
{status:10s}\n")
            if t.error_msg:
                self.tokens_text.insert(tk.END, f"      └ {t.error_msg}\n")

    def _update_tables(self):
        self.tables_text.delete(1.0, tk.END)
        if not self.shared.tokens:
            self.tables_text.insert(1.0, "Нет данных.")
            return
        out = ["=" * 80, "ТАБЛИЦЫ ЛЕКСЕМ", "=" * 80]
        out.append("\n1. КЛЮЧЕВЫЕ СЛОВА:")
        for w, i in sorted(self.shared.keywords.items(), key=lambda x: x[1])[:20]:
            out.append(f"  W{i:4d} : {w}")
        out.append("\n2. ИДЕНТИФИКАТОРЫ:")
        for n, i in sorted(self.shared.identifiers.items(), key=lambda x: x[1])[:20]:
            out.append(f"  I{i:4d} : {n}")
        out.append("\n3. ЧИСЛА:")
        for n, i in sorted(self.shared.numbers.items(), key=lambda x: x[1])[:20]:
            out.append(f"  N{i:4d} : {n}")
        self.tables_text.insert(1.0, '\n'.join(out))

    def _update_errors(self):
        self.errors_text.delete(1.0, tk.END)
        if self.shared.errors:
            for e in self.shared.errors[:50]:
                self.errors_text.insert(tk.END, f"{e}\n")
        else:
            self.errors_text.insert(1.0, "✅ Ошибок нет")

    def _update_stats(self):
        error_cnt = len([t for t in self.shared.tokens if t.lex_type == LexemType.ERROR])
```

```
        stats = {
            "Всего лексем:": len(self.shared.tokens),
            "Идентификаторов:": len(self.shared.identifiers),
            "Чисел:": len(self.shared.numbers),
            "Строк:": len(self.shared.strings),
            "Ключевых слов:": len([t for t in self.shared.tokens if t.lex_type ==
LexemType.KEYWORD]),
            "Операций:": len([t for t in self.shared.tokens if t.lex_type ==
LexemType.OPERATION]),
            "Ошибок:": error_cnt
        }
        for label, val in stats.items():
            if label in self.stats_labels:
                color = "red" if label == "Ошибок:" and val > 0 else "blue"
                self.stats_labels[label].config(text=str(val), foreground=color)


class RPNConverterGUI:
    def __init__(self, parent, shared, lexer_app, default_font):
        self.parent = parent
        self.shared = shared
        self.lexer_app = lexer_app
        self.default_font = default_font
        self.converter = OPZConverter()
        self.history = []
        self.setup_ui()

    def setup_ui(self):
        main = ttk.Frame(self.parent, padding="15")
        main.pack(fill=tk.BOTH, expand=True)
        main.columnconfigure(0, weight=1)
        main.columnconfigure(1, weight=1)
        main.rowconfigure(1, weight=1)

        toolbar = ttk.Frame(main)
        toolbar.grid(row=0, column=0, columnspan=2, sticky="ew", pady=(0, 15))
        ttk.Button(toolbar, text="Открыть файл", command=self.open_file).pack(side=tk.LEFT,
padx=5)
        ttk.Button(toolbar, text="Автоперевод", command=self.auto_convert).pack(side=tk.LEFT,
padx=5)
        ttk.Button(toolbar, text="Пошагово", command=self.start_step).pack(side=tk.LEFT, padx=5)
        ttk.Button(toolbar, text="Очистить", command=self.clear_all).pack(side=tk.LEFT, padx=5)

        ttk.Label(toolbar, text="Примеры:").pack(side=tk.LEFT, padx=(20, 5))
        self.example_var = tk.StringVar()
        self.example_combo = ttk.Combobox(
            toolbar,
            textvariable=self.example_var,
            values=[
                "1. Простые присваивания и арифметика",
                "2. Массивы",
                "3. Вызов функций",
                "4. Смешанный (арифметика + массивы)",
                "5. Тернарный оператор ifelse",
                "6. Условный оператор if-else",
                "7. Цикл while",
                "8. Объявление функции",
                "9. Смешанный (всё вместе)"
            ],
            state="readonly",
            width=40
        )
        self.example_combo.pack(side=tk.LEFT, padx=5)
        ttk.Button(toolbar, text="Загрузить",
command=self.load_selected_example).pack(side=tk.LEFT, padx=5)

        left = ttk.LabelFrame(main, text="Исходный код на R", padding="10")
        left.grid(row=1, column=0, sticky="nsew", padx=(0, 10))
```

```
        left.columnconfigure(0, weight=1)
        left.rowconfigure(0, weight=1)
        self.code_text = scrolledtext.ScrolledText(left, font=self.default_font)
        self.code_text.grid(row=0, column=0, sticky="nsew")

        right = ttk.Frame(main)
        right.grid(row=1, column=1, sticky="nsew")
        right.columnconfigure(0, weight=1)
        right.rowconfigure(0, weight=1)
        right.rowconfigure(1, weight=1)

        sf = ttk.LabelFrame(right, text="Стек операций", padding="10")
        sf.grid(row=0, column=0, sticky="nsew", pady=(0, 10))
        sf.columnconfigure(0, weight=1)
        sf.rowconfigure(0, weight=1)
        self.stack_text = scrolledtext.ScrolledText(sf, font=self.default_font, bg='#f0f0f0')
        self.stack_text.grid(row=0, column=0, sticky="nsew")

        of = ttk.LabelFrame(right, text="Обратная польская запись", padding="10")
        of.grid(row=1, column=0, sticky="nsew")
        of.columnconfigure(0, weight=1)
        of.rowconfigure(0, weight=1)
        self.output_text = scrolledtext.ScrolledText(of, font=self.default_font)
        self.output_text.grid(row=0, column=0, sticky="nsew")

        cf = ttk.LabelFrame(main, text="Пошаговый режим", padding="10")
        cf.grid(row=2, column=0, columnspan=2, sticky="ew", pady=(15, 0))

        self.step_btns = {}
        for name, txt, cmd in [('reset', 'Начало', self.step_reset),
                               ('back', 'Назад', self.step_back),
                               ('forward', 'Вперед', self.step_forward),
                               ('end', 'Конец', self.step_end)]:
            btn = ttk.Button(cf, text=txt, command=cmd, state=tk.DISABLED)
            btn.pack(side=tk.LEFT, padx=5)
            self.step_btns[name] = btn

        self.step_info = ttk.Label(cf, text="", font=self.default_font)
        self.step_info.pack(side=tk.LEFT, padx=20)

        self.status = ttk.Label(main, text="Готов", relief=tk.SUNKEN, anchor=tk.W)
        self.status.grid(row=3, column=0, columnspan=2, sticky="ew", pady=(10, 0))

        self.example_var.set("1. Простые присваивания и арифметика")
        self.load_selected_example()

    def get_examples(self):
        return {
            "1. Простые присваивания и арифметика": """# Простые присваивания и арифметика
z ← x + y * 2
w ← (x + y) * (z - 5) / 3
a ← b + c - 100
result ← x + y - z * w / 2""",

            "2. Массивы": """# Работа с массивами
arr1 ← a[1, 2, 3]
arr2 ← b[i, j+1, k-2]
arr3 ← matrix[1, 1] + matrix[2, 2]
result ← arr[1, 2, g+4] + c""",

            "3. Вызов функций": """# Вызов функций
x ← sum(1, 2, 3)
y ← c(1, 2, 3, 4, 5)
z ← g()
result ← f(2, y2, x) / 123 + g()""",

            "4. Смешанный (арифметика + массивы)": """# Смешанный пример
y1 ← a1 * (b + c) - d / (e - 1)
```

```
y2 ← a[1, 2, g+4] + c
z3 ← f(2, y2, x) / 123 + g()
result ← y1 + y2 * z3""",

            "5. Тернарный оператор ifelse": """# Тернарный оператор ifelse
z ← ifelse(x > y, x + 1, y - 1)
result ← ifelse(z > 10, z * 2, z / 2)
t4 ← hi + ifelse(a > b, b + 1, x + i - 2)
h5 ← ifelse(f > 5, f, 5)""",

            "6. Условный оператор if-else": """# Условный оператор if-else
if (x > 10) {
    x ← x - 1
    y ← y + 1
}

if (a > b) {
    x ← 1; y ← 2
} else {
    x ← 3
}""",

            "7. Цикл while": """# Цикл while
while (a > b) {
    x ← x - 1
    y ← y - 1
    z ← z + 1
}

while (i < 10) {
    sum ← sum + i
    i ← i + 1
}""",

            "8. Объявление функции": """# Объявление функции
f ← function() {
    x ← x + 1
    y ← y + 1
}""",

            "9. Смешанный (всё вместе)": """# Пример для перевода в ОПЗ (все конструкции)
g ← function() {
    x ← x + 1
    y ← y[5, 7, d]
    u ← i + 5
}

if (a > b) {
    x ← h(x, 6)
    y ← t
} else {
    x ← 2
    y ← z + ifelse(f > 5, f, 5)
}

while (x > 0) {
    x ← f(x - 1, y, z[5])
    y ← y + 1
}"""
        }

    def load_selected_example(self):
        selected = self.example_var.get()
        examples = self.get_examples()
        if selected in examples:
            self.code_text.delete(1.0, tk.END)
            self.code_text.insert(1.0, examples[selected])
            self.status.config(text=f"Загружен пример: {selected}", foreground="green")
```

```
def open_file(self):
    fn = filedialog.askopenfilename(filetypes=[("R files", "*.r"), ("All", "*.*")])
    if fn:
        with open(fn, 'r', encoding='utf-8') as f:
            self.code_text.delete(1.0, tk.END)
            self.code_text.insert(1.0, f.read())
        self.status.config(text=f"Загружен: {Path(fn).name}", foreground="green")

def run_lexical(self, code):
    lexer = RLexer()
    self.shared.tokens = lexer.tokenize(code)
    self.shared.errors = lexer.errors.copy()
    self.shared.identifiers = lexer.identifiers.copy()
    self.shared.numbers = lexer.numbers.copy()
    self.shared.strings = lexer.strings.copy()
    self.shared.comments = lexer.comments.copy()
    self.shared.keywords = lexer.keywords.copy()
    self.shared.delimiters = lexer.delimiters.copy()
    self.shared.operations = lexer.operations.copy()
    self.lexer_app.update()

def auto_convert(self):
    code = self.code_text.get(1.0, tk.END)
    if not code.strip():
        messagebox.showwarning("", "Нет кода!")
        return
    try:
        self.run_lexical(code)
        significant = [t for t in self.shared.tokens if t.lex_type != LexemType.COMMENT]
        self.converter = OPZConverter()
        rpn, self.history = self.converter.convert_expression(significant)
        self.output_text.delete(1.0, tk.END)
        self.output_text.insert(1.0, rpn)
        self.status.config(text="Перевод завершён", foreground="green")
        for btn in self.step_btns.values():
            btn.config(state=tk.DISABLED)
        self.step_info.config(text="")
    except Exception as e:
        messagebox.showerror("Ошибка", str(e))

def start_step(self):
    code = self.code_text.get(1.0, tk.END)
    if not code.strip():
        messagebox.showwarning("", "Нет кода!")
        return
    try:
        self.run_lexical(code)
        significant = [t for t in self.shared.tokens if t.lex_type != LexemType.COMMENT]
        self.converter = OPZConverter()
        rpn, self.history = self.converter.convert_expression(significant)
        self.converter.load_history(self.history)
        for btn in self.step_btns.values():
            btn.config(state=tk.NORMAL)
        self.converter.current_step = 0
        self.update_display()
        self.status.config(text="Пошаговый режим", foreground="blue")
    except Exception as e:
        messagebox.showerror("Ошибка", str(e))

def update_display(self):
    state = self.converter.get_current_state()
    if state:
        self.stack_text.delete(1.0, tk.END)
        if state['stack']:
            self.stack_text.insert(1.0, "Стек (верх → низ):\n" + "─" * 35 + "\n")
            for op in reversed(state['stack']):
                self.stack_text.insert(tk.END, f"│ {str(op)[:31]:^31} │\n")
```

```
                self.stack_text.insert(tk.END, "—" * 35)
            else:
                self.stack_text.insert(1.0, "Стек пуст\n" + "—" * 35)

            self.output_text.delete(1.0, tk.END)
            if state['output']:
                self.output_text.insert(1.0, ' '.join(state['output']))

            token = state.get('current_token', '')
            action = state.get('action', '')
            total = len(self.history)
            current = state['step'] + 1 if state['step'] ≥ 0 else 0
            self.step_info.config(text=f"Шаг {current}/{total} | {action} | Токен: {token}")

            self.step_btns['back'].config(state=tk.NORMAL if self.converter.current_step > 0 else
tk.DISABLED)
            self.step_btns['forward'].config(state=tk.NORMAL if self.converter.current_step <
len(self.history) - 1 else tk.DISABLED)
            self.step_btns['reset'].config(state=tk.NORMAL if self.converter.current_step > 0
else tk.DISABLED)
            self.step_btns['end'].config(state=tk.NORMAL if self.converter.current_step <
len(self.history) - 1 else tk.DISABLED)

    def step_forward(self):
        if self.converter.step_forward():
            self.update_display()

    def step_back(self):
        if self.converter.step_backward():
            self.update_display()

    def step_reset(self):
        self.converter.reset_to_start()
        self.update_display()

    def step_end(self):
        self.converter.go_to_end()
        self.update_display()

    def clear_all(self):
        self.code_text.delete(1.0, tk.END)
        self.output_text.delete(1.0, tk.END)
        self.stack_text.delete(1.0, tk.END)
        self.converter.reset_all()
        for btn in self.step_btns.values():
            btn.config(state=tk.DISABLED)
        self.step_info.config(text="")
        self.status.config(text="Готов", foreground="green")

        self.shared.tokens = []
        self.shared.errors = []
        self.shared.identifiers = {}
        self.shared.numbers = {}
        self.shared.strings = {}
        self.shared.comments = {}
        self.shared.keywords = {}
        self.shared.delimiters = {}
        self.shared.operations = {}
        self.lexer_app.update()


class CombinedApplication:
    def __init__(self, root):
        self.root = root
        self.root.title("Лексический анализатор R + перевод в ОПЗ")
        self.root.geometry("1600x900")

        self.default_font = ('JetBrains Mono', 12)
```

```
        style = ttk.Style()
        style.configure('.', font=self.default_font)
        style.configure('TLabel', font=self.default_font)
        style.configure('TButton', font=self.default_font)
        style.configure('TFrame', font=self.default_font)
        style.configure('TLabelframe', font=self.default_font)
        style.configure('TLabelframe.Label', font=self.default_font)
        style.configure('TNotebook', font=self.default_font)
        style.configure('TNotebook.Tab', font=self.default_font)
        style.configure('TCombobox', font=self.default_font)

        self.shared = SharedData()

        self.notebook = ttk.Notebook(root)
        self.notebook.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

        self.rpn_frame = ttk.Frame(self.notebook)
        self.notebook.add(self.rpn_frame, text="Перевод в ОПЗ")

        self.lexer_frame = ttk.Frame(self.notebook)
        self.notebook.add(self.lexer_frame, text="Лексический анализатор")
        self.lexer_app = LexicalAnalyzerGUI(self.lexer_frame, self.shared, self.default_font)

        self.rpn_app = RPNConverterGUI(self.rpn_frame, self.shared, self.lexer_app,
self.default_font)


def main():
    root = tk.Tk()
    app = CombinedApplication(root)
    root.mainloop()


if __name__ == "__main__":
    main()
```

Вывод

Реализация в файле polska.py является расширением классического алгоритма Дейкстры. Она учитывает особенности языка R, включая функции, массивы и управляющие конструкции. Алгоритм корректно преобразует входную последовательность токенов в ОПЗ.